

Control Unit

Function

Gets the 8 bits instruction and emits the suitable control signals for each block. It decides the stalls from the branches and the interrupts.

The FSM is RUN in normal (00), when getting interrupt it turns to SAVE_PC (01) which saves the current pc to the memory and the current flags in the status register, when getting pc_saved it turns to LOAD_INTERRUPT (10) which loads the interrupt into the PC for only one cycle then goes to RUN_INTERRUPT (11) till gets an RTI instruction then goes to RUN.

Inputs

- rst
- Interrupt
- pc_saved
- Instruction[7:0]

Outputs

- write_reg_en
- write_reg_address_sel[1:0] (0 means ra, 1 means rb, 2 means R3)
- write_reg_data_sel (0 means from ALU, 1 means from Memory)
- read_reg_a_sel (0 means ra, 1 means R3)
- read_reg_b_sel (0 means rb, 1 means R3)
- flush_1_instrucion (LDM, LDD, STD)
- flush_2_instrucions (LOOP, conditional branches)
- flush_3_instructions (CALL, RET, RTI, interrupt)
- push_pc
- pop_pc
- current_next_pc_sel (when 1 current (interrupt) and when 0 next (normal))
- pc_saving (high in the case of PC_SAVE)
- alu_op[3:0]
- alu_B_sel[1:0] (0 means da, 1 means db, 2 means immediate)
- flag_en
- flag_address[1:0]
- current_flag (0 if using the flag out from CCR, 1 if using the direct zero flag from the ALU (LOOP))
- store_flags (from CCR to status)
- load_flags (from status to CCR)
- pc_load_en

- pc_load_data_sel (0 means db, 1 means ISR)
- mem_wr_en
- mem_interface_sel[1:0] (0 means SP, 1 means instruction, 2 means interrupt_instruction, 3 means data)
- write_to_reg (high when the instruction is writing into reg)
- destination_addr[1:0] (if the instruction is writing into reg, this is its address)
- use_memory (high when the instruction uses the memory)

ALU_OP

- ADD = 0
- SUB = 1
- AND = 2
- OR = 3
- RLC B = 4
- RRC B = 5
- SETC = 6
- CLRC = 7
- NOT B = 8
- NEG B = 9
- INC B = 10
- DEC B = 11
- bypass B = 12